GRAPH NEURAL NETWORKS FOR SYSTEM CALL GRAPH-BASED INTRUSION DETECTION

Raphaël Faure & Tristan Beruard & Sacha
CentraleSupélec
{raphael.faure2, tristan.beruard}@student-cs.fr

ABSTRACT

Host-based intrusion detection relies on identifying deviations in the low-level behavior of processes. System call traces are a natural signal for this task because they describe the interaction between a program and the operating system at a fine granularity. In this project, we model each trace as a directed weighted syscall graph and compare classical graph scoring with graph neural networks on ADFA-LD Creech & Hu (2013) and two LID-DS scenarios et al. (2021). Our benchmark includes a PageRank-based anomaly detector inspired directly by our exploratory notebook, standard graph backbones such as GCN Kipf & Welling (2017), GraphSAGE, and GAT, and a weighted GCN variant denoted WGCN+. The results show that graph representations are informative on both datasets, but that the ranking of models depends strongly on the scenario: GAT performs best on ADFA-LD, while the PageRank baseline remains highly competitive on the bruteforce LID-DS scenario.

1 INTRODUCTION/MOTIVATION

Host-based intrusion detection remains difficult because malicious activity often appears as a subtle deformation of otherwise legitimate execution traces. System call logs are attractive in this setting because they are available at the operating-system boundary and provide a detailed view of process behavior. However, a purely sequential representation can make it difficult to distinguish a harmless local reordering from a structural change in the transition pattern of the process.

Our starting point is that a syscall trace is not only a sequence, but also a transition system. If two syscalls interact repeatedly, if a subset of calls becomes a hub, or if unusual loops emerge, this structure is naturally visible in a graph representation. This motivates our main research questions: does a syscall-graph representation improve host-based intrusion detection, which graph models are the most relevant for these data, and how stable are these conclusions across datasets and attack scenarios?

The practical motivation is straightforward. Better host-based intrusion detection can support endpoint protection, anomaly triage, incident response, and post-compromise forensics. From a machine learning perspective, the project also provides a useful case study for comparing interpretable graph scoring methods with learned graph representations on a real cybersecurity task.

2 PROBLEM DEFINITION

Let a syscall trace be a sequence $s = (x_1, \dots, x_T)$, where each token x_t is either an integer syscall identifier, as in ADFA-LD, or a syscall name extracted from a raw trace, as in LID-DS. From each sequence we build a directed weighted graph $G = (V, E, w)$ where each node in V is a unique syscall and each edge $(u, v) \in E$ records the number of times syscall u is immediately followed by syscall v in the trace.

The supervised task is binary graph classification. Given a trace graph G_i and its label $y_i \in \{0, 1\}$, with 0 denoting normal behavior and 1 an attack trace, we seek a predictor $f(G_i) \to \hat{y}_i$. Because the datasets are imbalanced and scenario-dependent, we evaluate the predictor with accuracy, precision, recall, F1 score, and ROC-AUC rather than relying on accuracy alone.

We also consider an anomaly-detection baseline based on the PageRank notebook. In that setting, the training procedure builds a normal library from benign traces only and the decision is taken from an anomaly score computed on sliding windows. The problem then becomes threshold-based anomaly scoring instead of direct discriminative classification.

3 Related Work

System call based intrusion detection has historically been studied through sequence models and symbolic matching rules because the data naturally arrives as an ordered stream. Graph-based approaches argue that transition structure itself is discriminative and should therefore be modeled explicitly. The system-call graph formulation of Various (2018) is particularly relevant here because it motivates representing a trace by its directed transition graph rather than by sequence statistics only.

On the representation-learning side, graph neural networks provide a natural way to combine local transition structure with node-level attributes. GCN Kipf & Welling (2017) is a standard baseline based on neighborhood aggregation, while GraphSAGE and GAT extend this idea respectively through flexible aggregation and attention over neighbors. Although these families are widely used in graph learning, they remain comparatively underexplored for host-based intrusion detection on syscall graphs.

Our classical baseline is related to the PageRank anomaly scoring strategy of Unknown (2013). The original idea is to derive transition suspiciousness from a PageRank-weighted normal graph and then measure the distance between observed windows and a library of normal patterns. Our implementation keeps this core mechanism but adapts it to a multi-trace training pipeline and to reproducible Hydra-based experiments.

4 Methodology

Datasets and preprocessing. We use two host-based intrusion detection datasets. ADFA-LD Creech & Hu (2013) provides integer-encoded syscall traces with separate normal and attack folders. LID-DS et al. (2021) provides richer per-sample archives containing raw `.sc` traces and metadata. For LID-DS, our final study focuses on the scenarios `Bruteforce_CWE-307` and `CVE-2014-0160`. To stay consistent with the exploratory notebook, our LID-DS pipeline keeps the original `test` traces and performs a local stratified train/validation/test split on that subset.

Graph construction and node features. Each trace is converted into a directed weighted graph. For ADFA-LD, node features include relative frequency, in/out degree, weighted degree, PageRank, and self-loop indicators. For LID-DS, we enrich the representation with process diversity, return-value statistics, and temporal gap statistics recovered from the raw traces. Each graph is then turned into a PyTorch Geometric object with both numeric node features and a learned embedding for the syscall identity.

PageRank baseline. The PageRank detector follows the same core logic as our exploratory notebook implementation. We build a normal transition graph, compute PageRank scores, derive a suspiciousness map $k(u, v)$ from the weighted outgoing contribution of each transition, slide a fixed-size window over the observed sequence, and score each observed window by its minimum weighted distance to a normal pattern. The final anomaly score is the proportion of windows whose distance exceeds a threshold.

Relative to the notebook, the production implementation makes three engineering adaptations. First, the pattern library is built from all normal training traces instead of a single file. Second, we use pattern subsampling, caching, and prefix-based candidate filtering to keep evaluation tractable on larger datasets. Third, the distance and anomaly thresholds are selected on the validation split rather than fixed manually. The mathematical core of the notebook is therefore preserved while the training protocol is generalized to our datasets.

GNN models. We compare four graph neural architectures. The plain GCN baseline applies edge-weighted GCN layers to the concatenation of learned syscall embeddings and numeric node features.

Dataset	Graphs	Attack %	Mean $\|V\|$	Mean $\|E\|$	Mean density	Mean seq. len.
ADFA-LD	5951	12.54	19.94	57.29	0.24	461.70
LID-DS (Bruteforce_CWE-307)	907	15.33	29.47	240.53	0.28	8136.46
LID-DS (CVE-2014-0160)	878	13.67	29.78	113.81	0.13	1640.26

Table 1: Graph-level inspection of the datasets used by the training pipeline. For LID-DS, the inspection follows the same selection as the experiments and therefore focuses on the original `test` traces before the local stratified re-split.

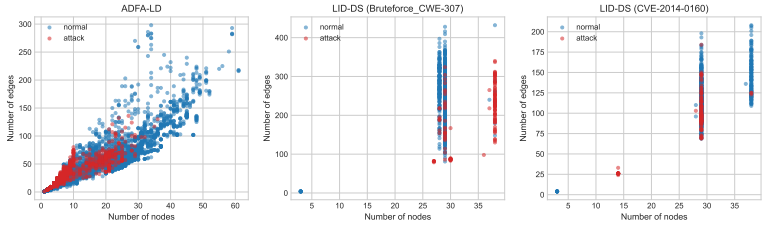

Figure 1: Number of nodes versus number of edges for the generated syscall graphs, colored by class.

GraphSAGE replaces the convolution operator with mean aggregation over neighbors. GAT adds attention coefficients over incoming messages. Finally, our main architecture, denoted **WGCN+**, keeps weighted GCN propagation but adds layer normalization after each message-passing layer and uses hybrid graph pooling obtained by concatenating mean pooling and max pooling. This architecture is designed for small-to-medium syscall graphs where both average transition behavior and strong localized motifs can be informative.

Training protocol. All GNN models are trained with Adam and selected using validation F1. The best validation checkpoint is then evaluated on the held-out test split. The report figures and tables are exported automatically from the training pipeline so that the manuscript remains synchronized with the actual experimental artifacts.

4.1 Graph Inspection

The generated syscall graphs remain in a small-to-medium regime while staying fairly dense: ADFA-LD contains 5951 graphs with 19.9 nodes and 57.3 edges on average (directed density 0.245); LID-DS (Bruteforce_CWE-307) contains 907 graphs with 29.5 nodes and 240.5 edges on average (directed density 0.283); LID-DS (CVE-2014-0160) contains 878 graphs with 29.8 nodes and 113.8 edges on average (directed density 0.132). This profile makes local message-passing models appropriate. We therefore compare a plain GCN baseline, GraphSAGE for more robust neighborhood aggregation, GAT to test whether attention over heterogeneous syscall transitions improves discrimination, and our WGCN+ variant that combines weighted GCN propagation, learned syscall embeddings, structural node features, layer normalization, and hybrid graph pooling.

5 Evaluation

Experimental setting. The benchmark combines PageRank, GCN, GraphSAGE, GAT, and WGCN+ on ADFA-LD. On LID-DS, we report PageRank and WGCN+ on the two retained scenarios. The third scenario initially considered, `CWE-89-SQL-injection`, was removed from the final study because preprocessing and graph construction on this scenario exceeded the available memory budget in our environment.

Main observations. Three conclusions stand out from the benchmark. First, on ADFA-LD, all GNN variants clearly outperform the PageRank baseline in ROC-AUC and F1, and GAT achieves the best overall test performance. This suggests that, on ADFA-LD, heterogeneous transition importance matters and is captured well by attention. Second, on the LID-DS bruteforce scenario, both approaches are very strong, but the PageRank baseline is actually the best method in our experi-

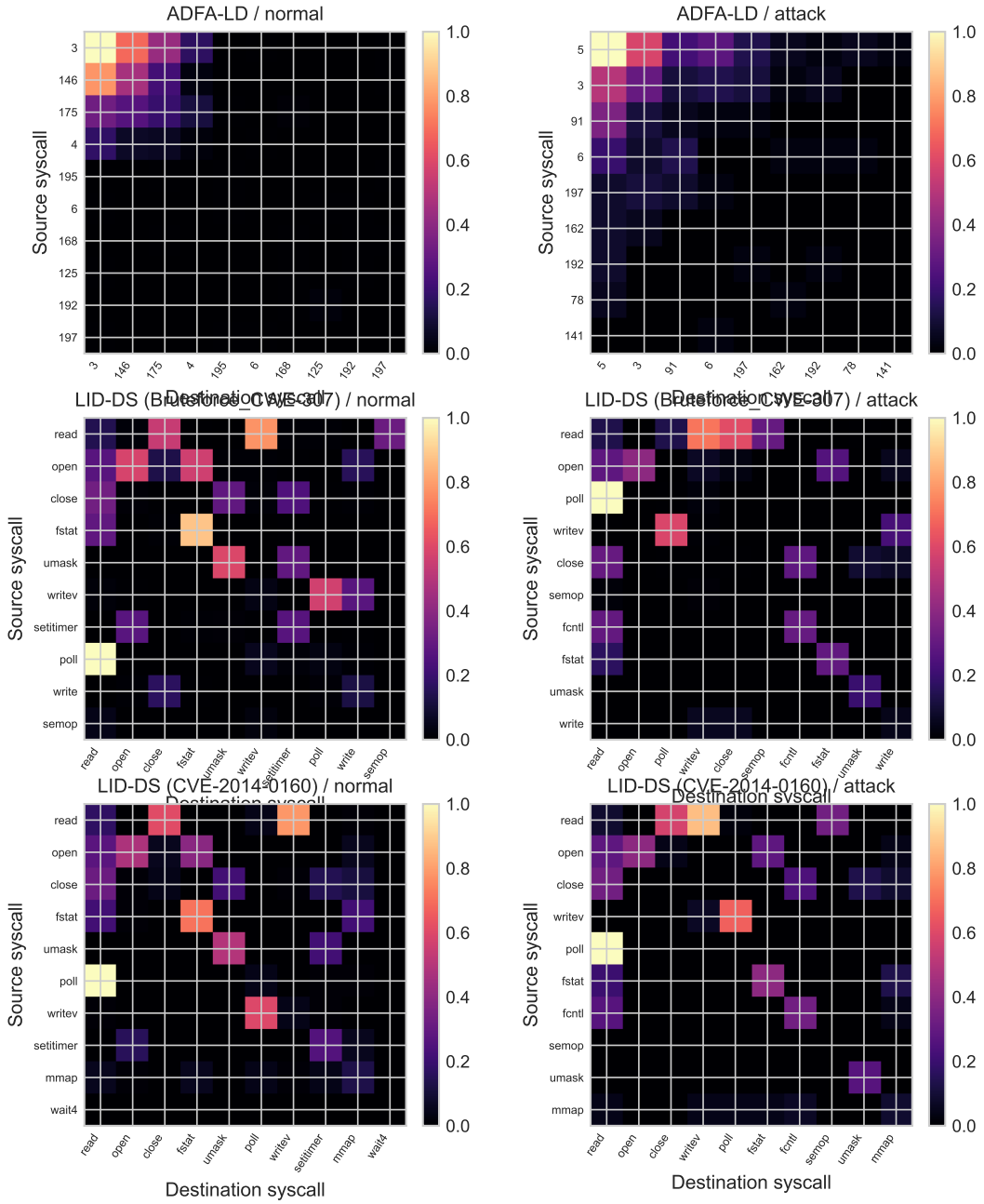

Figure 2: Representative transition matrices selected near the mean graph profile for each dataset/scenario and class. Each panel shows the normalized weighted adjacency restricted to the most central syscalls of the selected trace. On ADFA-LD, the axis labels are the raw syscall identifiers provided by the dataset; on LID-DS, they are syscall names parsed from the `.sc` traces.

ments, which indicates that this scenario is dominated by highly regular local transition anomalies that can already be captured by the window-based normal-pattern library. Third, on the harder `CVE-2014-0160` scenario, PageRank remains imperfect but usable, whereas WGCN+ reaches high ROC-AUC yet collapses to zero F1 under the default classification threshold. This means the model still ranks traces reasonably well, but its final decision boundary is poorly calibrated under this scenario.

Dataset	Scenario	Model	Acc.	F1	Prec.	Rec.	ROC-AUC
adfa_ld	all	pagerank	0.8128	0.4652	0.3619	0.6510	0.8092
adfa_ld	all	gnn:wgcn_plus	0.9244	0.7289	0.6612	0.8121	0.9636
adfa_ld	all	gnn:gcn	0.9362	0.7164	0.8067	0.6443	0.9567
adfa_ld	all	gnn:graphsage	0.9337	0.7208	0.7612	0.6846	0.9617
adfa_ld	all	gnn:gat	0.9488	0.7987	0.7857	0.8121	0.9790
lid_ds	Bruteforce_CWE-307	pagerank	0.9890	0.9630	1.0000	0.9286	1.0000
lid_ds	Bruteforce_CWE-307	gnn:wgcn_plus	0.9560	0.8462	0.9167	0.7857	0.9666
lid_ds	CVE-2014-0160	pagerank	0.8864	0.3750	0.7500	0.2500	0.7818
lid_ds	CVE-2014-0160	gnn:wgcn_plus	0.8636	0.0000	0.0000	0.0000	0.9380

Table 2: Test-set metrics exported automatically from the training pipeline.

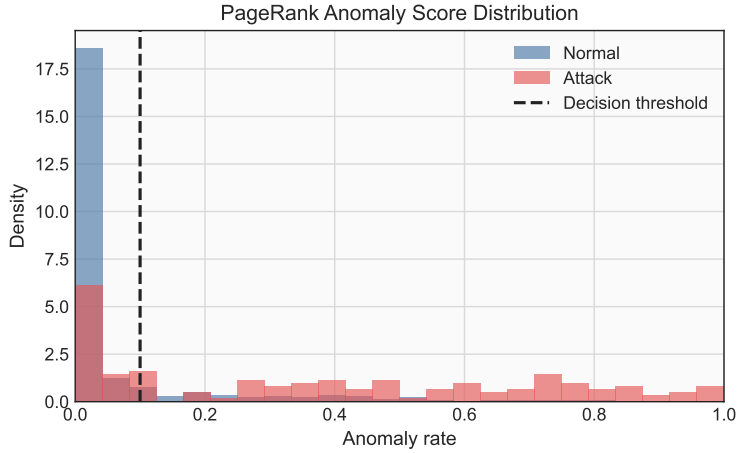

Figure 3: Diagnostic plot for `adfa_ld_pagerank`: distribution of test anomaly scores with the decision threshold.

Interpretation and limitations. The graph inspection and the benchmark together suggest that there is no single winner across all operating conditions. Dense and repetitive scenarios such as bruteforce attacks are particularly favorable to the PageRank detector because its scoring rule is explicitly based on deviations from frequent normal transition patterns. By contrast, ADFA-LD benefits more from learned graph representations, especially attention-based ones. WGCN+ remains useful as a structured in-house architecture because it combines weighted propagation with feature-rich node descriptions and remains competitive across datasets, even when it is not always the best model. The current study nevertheless remains limited by CPU-only training, by the exclusion of the SQL injection scenario, and by the absence of a learned threshold-calibration stage for GNN outputs.

5.1 Automated Results

5.2 Diagnostics: adfa_ld_pagerank

5.3 Diagnostics: adfa_ld_gat

6 Conclusions

This project shows that syscall graphs form a useful and operational representation for host-based intrusion detection. The graph construction step preserves transition structure while enabling both classical graph scoring and graph neural message passing. In our experiments, GAT provides the strongest results on ADFA-LD, whereas the PageRank baseline remains remarkably effective on the bruteforce LID-DS scenario. The main lesson is therefore not that one model dominates everywhere, but that graph-based modeling exposes structure that different families of methods can exploit in complementary ways.

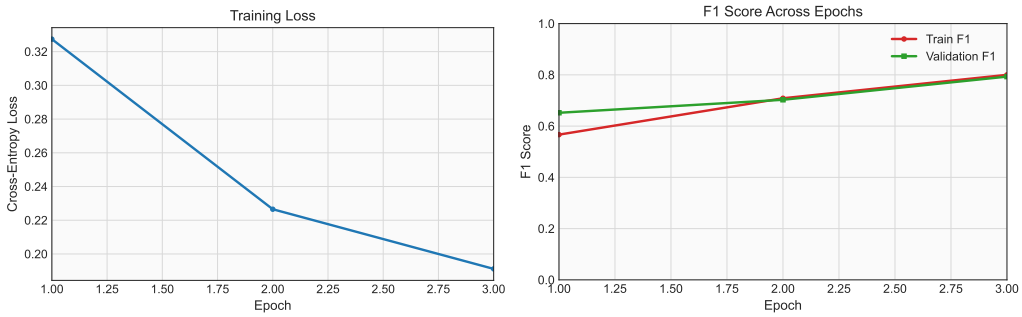

Figure 4: Training diagnostics for `adfa_ld_gat`: cross-entropy loss and F1 score as a function of epoch.

Several extensions follow naturally from this work. A first direction is to calibrate the GNN decision threshold on the validation split in the same spirit as the PageRank baseline, especially for difficult imbalanced scenarios such as `CVE-2014-0160`. A second direction is to include the SQL injection scenario once a more memory-efficient preprocessing path is available. Finally, richer dynamic or provenance-aware graph constructions could help incorporate longer-range temporal context beyond immediate syscall transitions.

7 References

References

Gideon Creech and Jiankun Hu. Adfa-ld: A new dataset for host-based intrusion detection. 2013.

Kiss et al. A modern and sophisticated host-based intrusion detection dataset. 2021.

Thomas Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *ICLR*, 2017.

Unknown. An anomaly intrusion detection method based on pagerank algorithm. *IEEE*, 2013.

Various. Intrusion detection on system call graphs. 2018.